

RAPPORT TECHNIQUE FINAL

Smart Fleet

Système de gestion de flotte intelligent
avec maintenance prédictive par IA



Gestion de flotte



Machine Learning



Suivi GPS temps réel



Tableaux de bord analytiques



Notifications temps réel

AUTEUR

Khalil Ebbouh

STACK BACKEND

Laravel 11 · PHP 8.2 · MySQL

MODULE IA

FastAPI · scikit-learn · RandomForest

DATE

Mai 2026

STACK FRONTEND

Angular 18 · TypeScript · Leaflet

TEMPS RÉEL

Pusher WebSockets · Laravel Sanctum

Table des matières

1	Introduction et contexte du projet	3
1.1	Problématique	3
1.2	Objectifs et périmètre	3
2	Architecture globale du système	4
2.1	Vue d'ensemble	4
2.2	Technologies utilisées	4
3	Backend Laravel — API REST	5
3.1	Authentification et gestion des rôles	5
3.2	Endpoints API	5
3.3	Services métier	6
4	Modèle de données	7
4.1	Schéma des tables principales	7
4.2	Relations entre entités	8
5	Module IA — Maintenance prédictive	9
5.1	Conception du modèle ML	9
5.2	Microservice FastAPI	9
5.3	Intégration Laravel ↔ ML service	10
5.4	Widget dashboard Angular	10
6	Frontend Angular — Interface utilisateur	11
6.1	Architecture SPA et modules fonctionnels	11
6.2	Tableau de bord analytique	11
6.3	Suivi GPS temps réel	12
7	Temps réel et notifications	12
7.1	Architecture Pusher WebSockets	12
7.2	Règles métier automatisées (FleetRulesService)	13
8	Sécurité et gestion des accès	13

1 Introduction et contexte du projet

1.1 Problématique

La gestion d'une flotte de véhicules professionnels constitue un défi opérationnel complexe : suivi des interventions de maintenance, coordination des chauffeurs et des missions, surveillance des consommations de carburant, gestion des incidents, et surtout **anticipation des pannes** avant qu'elles surviennent. Les solutions traditionnelles reposent sur des tableurs ou des logiciels monolithiques qui ne permettent ni le suivi en temps réel, ni l'analyse prédictive.

Smart Fleet répond à cette problématique en proposant une plateforme web complète, modulaire et intelligente, combinant un système de gestion opérationnelle et un moteur de prédiction par apprentissage automatique.

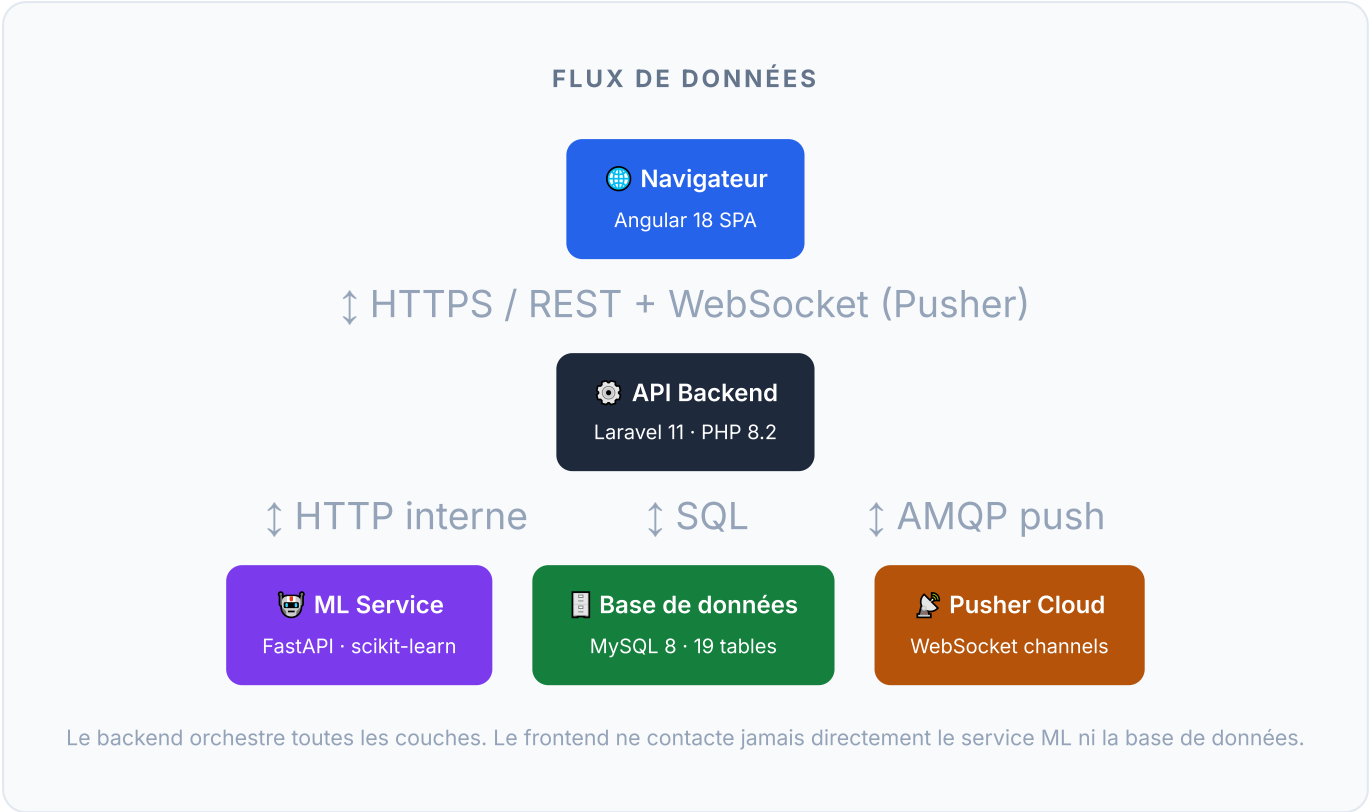
1.2 Objectifs et périmètre

11MODÈLES DE
DONNÉES**40+**ENDPOINTS API
REST**12**MODULES
ANGULAR**3**COUCHES
LOGICIELLES**Gestion du parc**Véhicules, conducteurs, maintenances,
carburant avec CRUD complet et photos.**Suivi GPS temps réel**Trajets enregistrés point par point, carte Leaflet
en direct, OSRM routing.**IA prédictive**Modèle RandomForest prédit les jours avant
maintenance par véhicule actif.**Alertes automatiques**Détection sur-vitesse, ralenti prolongé, mission
en retard — push WebSocket.**Analytique avancée**KPIs, graphiques ApexCharts — fuel,
maintenance, missions, statuts de flotte.**Emails transactionnels**Notifications mission, incident, maintenance avec
double mode SMTP (sandbox/live).

2 Architecture globale du système

2.1 Vue d'ensemble

Smart Fleet adopte une architecture **trois couches découplées** communiquant via des APIs REST JSON et des canaux WebSocket :



2.2 Technologies utilisées

COUCHE	TECHNOLOGIE	VERSION	RÔLE
Backend	Laravel	11.x	API REST, ORM Eloquent, Sanctum auth, scheduling
Backend	PHP	8.2	Langage serveur, types stricts, fibers
Backend	Laravel Sanctum	4.x	Tokens API Bearer, middleware auth
Frontend	Angular	18.0	SPA standalone components, signals, defer
Frontend	TypeScript	5.x	Typage strict, interfaces, generics
Frontend	Leaflet + OSRM	1.9	Cartes interactives, routing itinéraires
Frontend	ApexCharts	3.x	Graphiques analytiques du tableau de bord
IA	FastAPI	0.115	Microservice ML, validation Pydantic

COUCHE	TECHNOLOGIE	VERSION	RÔLE
IA	scikit-learn	1.x	RandomForestRegressor, sérialisation joblib
Infra	MySQL	8.0	Base de données relationnelle, 19 migrations
Infra	Pusher	Cloud	WebSocket temps réel — canaux privés
Infra	Mailtrap / Gmail	—	Emails transactionnels (sandbox + prod)

3

Backend Laravel — API REST

3.1 Authentification et gestion des rôles

L'authentification repose sur **Laravel Sanctum** avec des tokens Bearer. Chaque requête authentifiée porte un header `Authorization: Bearer {token}`. Le système définit **3 profils de rôle** avec des permissions granulaires :

FONCTIONNALITÉ	ADMIN	GESTIONNAIRE	CHAUFFEUR
Tableau de bord analytique complet	✓	✓	—
Gestion véhicules (lecture)	✓	✓	—
Gestion véhicules (écriture/suppression)	✓	—	—
Création/affectation de missions	✓	✓	—
Démarrer / terminer une mission	—	—	✓
Prédictions IA de maintenance	✓	✓	—
Alertes flotte temps réel	✓	✓	—
Gestion utilisateurs	✓	—	—
Déclarer un incident	✓	✓	✓

3.2 Endpoints API

L'API expose **40+ routes** RESTful organisées par domaine fonctionnel, toutes sécurisées par le middleware `auth:sanctum` :

DOMAINE	MÉTHODES PRINCIPALES	RÔLE REQUIS
Authentification	POST /login, POST /register, POST /logout	Public / Authentifié
Dashboard	GET /dashboard	Authentifié
Véhicules	GET /vehicles, GET /vehicles/{id}, POST, PUT, DELETE	Admin+Gest. (write: Admin)
Conducteurs	GET /drivers, POST /drivers, PUT, DELETE	Admin+Gest. (delete: Admin)
Missions	GET /missions, POST, PUT /start, PUT /complete	Rôle-dépendant
GPS / Trajets	POST /missions/{id}/trajets, GET /fleet/live	Chauffeur / Admin+Gest.
Maintenance	GET /maintenances, POST, PUT, DELETE	Admin+Gest. (write: Admin)

DOMAINE	MÉTHODES PRINCIPALES	RÔLE REQUIS
Carburant	GET /fuel-records, POST, PUT, DELETE	Admin+Gest. (write: Admin)
Incidents	GET /incidents, POST /incidents	Tous rôles
Alertes	GET /alerts, GET /fleet/alerts	Admin+Gest.
Prédictions IA	GET /predictions/maintenance, POST /refresh, GET /history	Admin+Gest.
Notifications	GET /notifications, POST /read, POST /read-all	Admin+Gest.
Emails	GET /email-logs	Admin+Gest.
Utilisateurs	GET /users, POST /role, DELETE	Admin

3.3 Services métier

La logique applicative est encapsulée dans **4 services Laravel** injectables :

MAINTENANCEPREDICTIONSERVICE

- `buildFeatures(Vehicle)` — extrait kilométrage, âge, historique
- `predict(Vehicle)` — appel HTTP au microservice FastAPI (timeout 5 s)
- `predictAndStore(Vehicle)` — prédit et sauvegarde en base

PUSHERSERVICE

- `notifyUser(userId, payload)` — canal `private-user.{id}`
- `publishMissionTrajet(missionId, payload)` — GPS en direct
- `publishFleetAlert(payload)` — `private-fleet.live`

EMAILSERVICE

- 10 types d'emails transactionnels (missions, incidents, maintenance, reset password)
- Double mode SMTP : Mailtrap sandbox ↔ Gmail/SMTP réel via toggle `MAIL_USE_REAL_INBOX`
- Log systématique dans `email_logs` (statut: sent / failed / skipped)

FLEETRULESERVICE

- `evaluate(Mission, Trajet, ?Trajet)` — déclenché à chaque point GPS
- Règle sur-vitesse : distance haversine / temps > 110 km/h
- Règle ralenti : immobilisation > 15 min dans rayon 40 m
- Règle retard : mission dépassant l'heure de fin prévue
- Déduplication 45 min : pas d'alerte répétée sur même règle



Scheduling automatique — La commande `predictions:check-maintenance` est planifiée via `withSchedule()` dans `bootstrap/app.php` pour s'exécuter chaque jour à 06h00. Elle rafraîchit automatiquement les prédictions pour tous les véhicules actifs sans intervention manuelle.

4 Modèle de données

4.1 Schéma des tables principales

🚗 vehicles		
id	bigint	PK
brand	varchar	Marque
model	varchar	Modèle
license_plate	varchar	Immatriculation
year	integer	Année
mileage	integer	Kilomètres
status	enum	active / en maintenance / inactive
photo_path	varchar	nullable

👤🚗 drivers		
id	bigint	PK
name	varchar	Nom complet
phone	varchar	Téléphone
license_number	varchar	Numéro permis
vehicle_id	bigint	FK → vehicles
photo_path	varchar	nullable

📋 missions		
id	bigint	PK
vehicle_id	bigint	FK → vehicles
title	varchar	Titre mission
status	enum	planned / in progress / completed
starts_at	datetime	Début prévu
ends_at	datetime	Fin prévue

🔧🚗 maintenance_predictions		
id	bigint	PK
vehicle_id	bigint	FK → vehicles
predicted_days	integer	Jours avant maintenance
predicted_date	date	Date prévue
urgency_level	enum	faible / moyen / élevé
features_snapshot	json	Features ML au moment du calcul
confidence_note	varchar	nullable
model_version	varchar	Version du modèle ML

📍 trajets (GPS)			⚠️ fleet_alerts		
id	bigint	PK	id	bigint	PK
mission_id	bigint	FK → missions	mission_id	bigint	FK → missions
lat	decimal	Latitude	type	enum	overspeed
lng	decimal	Longitude			vehicle_id delayed_n
recorded_at	datetime	Horodata GPS	message	text	Descriptio alerte
			meta	json	Données contextue (vitesse, etc.)
			acknowledged_at	datetime	nullable

4.2 Relations entre entités

MODÈLE	RELATIONS ELOQUENT
Vehicle	hasOne Driver · hasMany Maintenance · hasMany FuelRecord · hasMany MaintenancePrediction
Mission	belongsTo Vehicle · belongsToMany User · hasMany Trajet · hasMany Incident · hasMany FleetAlert
User	belongsToMany Mission
Trajet / FleetAlert / Incident	belongsTo Mission
EmailLog	morphTo related (Polymorphique : Mission, Incident, Maintenance, User...)

5 Module IA — Maintenance prédictive

5.1 Conception du modèle ML

Le moteur prédictif utilise un **RandomForestRegressor** (scikit-learn, 100 arbres de décision, `random_state=42`) entraîné sur un dataset de véhicules avec leurs historiques de maintenance.

FEATURES D'ENTRÉE (5)

- `mileage` — kilométrage total
- `km_par_jour` — utilisation journalière moyenne
- `km_depuis_derniere_maint` — km depuis dernière maintenance
- `age_vehicule_annees` — âge du véhicule
- `nb_maintenances_passees` — nombre d'interventions passées

SORTIE DU MODÈLE

- `jours_avant_maintenance` — entier 1–365
- `date_prevue` — date ISO calculée
- `niveau_urgence` — faible / moyen / élevé
- `confidence_note` — message explicatif



Logique d'urgence : < 7 jours → **Élevé** (intervenir cette semaine) · 7–14 jours → **Moyen** (planifier sous 2 semaines) · ≥ 15 jours → **Faible** (pas d'urgence immédiate). Le modèle est re-entraînable via `python train_model.py` et sérialisé dans `model.pkl`.

5.2 Microservice FastAPI

Le service ML tourne en tant que **processus indépendant** (port 8001 par défaut) et expose 3 endpoints :

ENDPOINT	MÉTHODE	DESCRIPTION
GET /	GET	Informations du service, liste des endpoints
GET /health	GET	État du service, modèle chargé, features attendues
POST /predict	POST	Calcule la prédiction de maintenance pour un véhicule

```
# Corps de requête (POST /predict) { "mileage": 87600, "km_par_jour": 240.0,
"km_depuis_derniere_maint": 19000, "age_vehicule_annees": 1, "nb_maintenances_passees": 15 } #
Réponse { "jours_avant_maintenance": 14, "date_prevue": "2026-06-11", "niveau_urgence": "moyen",
"confidence_note": "Planifier la maintenance dans les deux prochaines semaines." }
```

5.3 Intégration Laravel ↔ ML service

Le service **MaintenancePredictionService** encapsule tout le dialogue avec l'API ML. Il construit les features à partir du modèle Eloquent `Vehicle`, effectue l'appel HTTP (timeout 5 s, gestion d'erreur gracieuse si le service ML est hors ligne) et persiste le résultat dans `maintenance_predictions` avec un `features_snapshot` JSON pour traçabilité.

5.4 Widget dashboard Angular

Le composant `MaintenancePredictionWidgetComponent` (standalone, `ChangeDetectionStrategy.OnPush`, signals Angular 18) affiche en temps réel :

- La liste des prédictions triées par urgence (élevé → moyen → faible)
- Un badge compteur des véhicules urgents
- Photo du véhicule, immatriculation, nombre de jours restants, date en français, badge coloré
- Bouton « Actualiser » par ligne appelant `POST /predictions/maintenance/{id}/refresh`
- 3 états gérés : chargement (skeletons), erreur, vide

6

Frontend Angular — Interface utilisateur

6.1 Architecture SPA et modules fonctionnels

L'application utilise l'architecture **standalone components** d'Angular 18 (sans NgModule), avec @defer pour le chargement différé des graphiques et des sections secondaires. Les composants communiquent via **signals** Angular pour la réactivité fine-grain sans détection de changement globale.

MODULE	COMPOSANTS	FONCTIONNALITÉ
auth/	login, register, forgot-password, reset-password	Authentification complète avec gestion des tokens
dashboard/	dashboard, map-widget, prediction-widget	KPIs, graphiques, carte live, prédictions IA
vehicles/	vehicle-list, vehicle-form	CRUD véhicules avec upload photo
drivers/	driver-list, driver-form	CRUD conducteurs avec affectation véhicule
missions/	missions-list, mission-detail	Gestion et suivi des missions
trajets/	trajets-tracking, trajets-list	Affichage GPS temps réel, historique trajets
fleet/	fleet-live-map	Vue cartographique multi-véhicules en direct
maintenances/	maintenance-list, maintenance-form	Enregistrement des interventions
fuel/	fuel-list, fuel-form	Suivi consommation carburant
incidents/	incidents-list, report-incident	Déclaration et consultation des incidents
alerts/	alerts	Alertes flotte (sur-vitesse, retard, ralenti)
admin/	user-management, email-logs	Gestion utilisateurs et logs emails

6.2 Tableau de bord analytique

Le dashboard admin/gestionnaire présente une vue complète en grille sombre :

- **6 KPI cards** avec sparklines ApexCharts : total véhicules, véhicules actifs, missions du jour, litres carburant, alertes ouvertes, véhicules en maintenance
- **Carte Leaflet live** avec overlay de statistiques de flotte en temps réel
- **4 graphiques pleine largeur** (fuel area, maintenance bar, donut statuts, missions area, alertes bar, santé système)
- **Widget maintenance prédictive** (IA) — triée par urgence, actualisation à la demande
- **Colonne latérale** : conducteurs en ligne, missions récentes, top véhicules consommateurs

6.3 Suivi GPS temps réel

L'architecture temps réel GPS fonctionne en cascade : le chauffeur envoie des points via POST `/missions/{id}/trajets` → Laravel **FleetRulesService** évalue les règles et publie via Pusher → le frontend reçoit les événements WebSocket sur le canal `private-mission.{id}` et met à jour la position du marqueur avec une interpolation lisse (`smooth-marker.ts`) sur la carte Leaflet.



Routing OSRM : Les itinéraires sont calculés via le proxy GET `/api/osrm/route` qui relaie les requêtes vers le service OSRM public (`router.project-osrm.org`), permettant d'afficher le trajet optimal sur la route plutôt qu'une simple ligne droite.

7 Temps réel et notifications

7.1 Architecture Pusher WebSockets

Les communications temps réel reposent sur **Pusher** avec des canaux privés authentifiés. Le backend autorise chaque souscription via POST `/api/pusher/auth` (token Sanctum requis).

CANAL PUSHER	ÉVÉNEMENT	DÉCLENCHEUR	CONSOMMATEUR
private-user.{id}	notification	Mission assignée, incident, maintenance	Cloche notifications frontend
private-mission.{id}	trajet.created	POST <code>/missions/{id}/trajets</code>	Carte Leaflet suivi live
private-fleet.live	fleet.alert	Règle violée (vitesse/ralenti/retard)	Dashboard alertes flotte

7.2 Règles métier automatisées (FleetRulesService)

À chaque point GPS reçu, le **FleetRulesService** évalue trois règles de manière synchrone et publie une alerte WebSocket le cas échéant :



Sur-vitesse

Distance haversine entre deux points consécutifs / delta temps. Seuil : 110 km/h.



Ralenti prolongé

Véhicule immobile (<40 m) pendant plus de 15 minutes consécutives.



Mission en retard

Heure actuelle dépasse ends_at et mission encore active.



Anti-duplication

Fenêtre de 45 min : aucune alerte répétée pour la même règle/mission.

8 Sécurité et gestion des accès

La sécurité de l'application repose sur plusieurs couches complémentaires :



AUTHENTIFICATION

- Tokens Bearer Laravel Sanctum avec expiration configurable
- Stockage token en localStorage côté Angular



AUTORISATION

- Middleware role:admin,gestionnaire sur groupes de routes
- Middleware isAdmin pour les routes d'administration

- Guard Angular interceptant chaque route protégée
- Reset password par email avec token signé (1h)

- Vérification du rôle également dans les contrôleurs (double vérification)
- Canaux Pusher privés (auth backend obligatoire)



Redirection des non-authentifiés : Le middleware est configuré avec `redirectGuestsTo(fn () => null)` dans `bootstrap/app.php` pour retourner un JSON 401 au lieu de rediriger vers une route `login web`, évitant les erreurs 500 "Route [login] not defined" sur une API JSON pure.

9 Conclusion et perspectives

Smart Fleet est une plateforme de gestion de flotte **full-stack complète**, combinant des technologies modernes pour répondre aux besoins opérationnels réels des gestionnaires de parc automobile. Le système couvre l'ensemble du cycle de vie de la flotte, de la création d'un véhicule jusqu'à la prédiction automatique de sa prochaine maintenance.

Réalisations techniques clés

DOMAINE	RÉALISATION	TECHNOLOGIE
API REST sécurisée	40+ endpoints, 3 niveaux de rôles, tokens Sanctum	Laravel 11 · PHP 8.2
Modèle de données	19 migrations, 11 modèles Eloquent, relations polymorphiques	MySQL 8
Intelligence artificielle	RandomForest prédisant les jours avant maintenance sur 5 features	FastAPI · scikit-learn
Temps réel	GPS live, alertes flotte, notifications push	Pusher WebSockets
Interface utilisateur	SPA moderne, dashboard analytique, cartes interactives	Angular 18 · Leaflet · ApexCharts
Emails transactionnels	10 types d'emails loggés, double mode SMTP	Laravel Mail · Mailtrap
Sécurité	Auth Bearer, canaux privés, middleware rôles	Sanctum · Pusher Auth

Perspectives d'évolution

- **Application mobile** (React Native ou Flutter) pour les chauffeurs avec géolocalisation native
- **Amélioration du modèle ML** avec un dataset plus large et des features supplémentaires (météo, charge, type de route)
- **Re-entraînement automatique** du modèle à partir des données réelles accumulées dans la base
- **Rapports PDF automatiques** générés périodiquement (résumé mensuel de flotte)
- **Intégration IoT** : réception des données GPS directement depuis des boîtiers embarqués (OBD-II)
- **Module de facturation** : calcul des coûts d'exploitation par véhicule et par mission

Smart Fleet démontre qu'il est possible de concevoir et développer une plateforme de gestion industrielle complète avec des outils open-source modernes. L'intégration réussie de trois couches distinctes (API REST, microservice ML, SPA Angular) communiquant de manière découplée constitue l'un des principaux apports de ce projet.

La maintenance prédictive par IA représente une valeur ajoutée significative : en anticipant les pannes plusieurs semaines à l'avance, elle permet de réduire les arrêts non planifiés et d'optimiser les coûts de maintenance de la flotte.

-  Architecture trois couches découplées et scalable
-  Module IA opérationnel avec widget dashboard intégré
-  Temps réel GPS + alertes automatiques FleetRules
-  Système de rôles granulaire et sécurisé
-  Interface sombre moderne, responsive, multilingue (i18n)